

Case Study: Turf Management System using JSON Server (CRUD Operations)

Scenario

A sports academy wants to manage its **Turf Booking System**. Instead of storing data in memory, all booking details are stored in a **JSON Server**, which acts as a fake REST API.

The application should be modular, with each CRUD operation implemented in a separate JavaScript file.

Learning Objectives

Students will learn:

- JSON Server
- REST APIs
- GET, POST, PUT, DELETE
- Fetch API
- Async/Await
- JavaScript Modules (Import/Export)
- Error Handling

Folder Structure

```
turf-management/  
├── db.json  
├── package.json  
├── api/  
│   ├── getBookings.js  
│   ├── addBooking.js  
│   ├── updateBooking.js  
│   └── deleteBooking.js  
└── app.js
```

Step 1: Install JSON Server

```
npm init -y
```

```
npm install json-server
```

package.json

```
{
  "type": "module",
  "scripts": {
    "server": "json-server --watch db.json --port 3000",
    "start": "node app.js"
  }
}
```

Step 2: Create db.json

```
{
  "bookings": [
    {
      "id": 1,
      "customer": "Rahul",
      "sport": "Football",
      "hours": 2,
      "amount": 2400,
      "status": "Confirmed"
    },
    {
      "id": 2,
      "customer": "Priya",
      "sport": "Cricket",
      "hours": 3,
      "amount": 3600,
      "status": "Pending"
    },
    {
      "id": 3,
      "customer": "Arun",
      "sport": "Football",

```

```
    "hours": 1,  
    "amount": 1200,  
    "status": "Confirmed"  
  }  
]  
}
```